

C BEN KHELIL

| 2025 – 2026

## Travaux Pratiques N°3

# Héritage et Polymorphisme

*Java 1 : Fondamentaux de la POO*

### Objectifs du TP

À l'issue de ce TP, l'étudiant sera capable de :

- Objectif 1 : Maîtriser les concepts d'héritage et de hiérarchie de classes.
- Objectif 2 : Comprendre et utiliser correctement **super**.
- Objectif 3 : Appliquer les règles de redéfinition de méthodes.
- Objectif 4 : Gérer les packages et la visibilité.
- Objectif 5 : Implémenter le polymorphisme pour gérer une collection hétérogène d'objets.
- Objectif 6 : Appliquer la composition pour modéliser des périphériques complexes.

### Consignes

- Ce travail est **INDIVIDUEL**.
- Logiciel à utiliser **IntelliJ**.
- Il est fortement recommandé d'ouvrir le support du CM durant toute la séance du TP.

## 0 Approche de développement modulaire et organisation des packages

### 0.1 Introduction

Dans ce TP, vous développerez progressivement un système de gestion de parc informatique pour une entreprise technologique. Nous adopterons une approche de développement modulaire : le projet sera organisé en plusieurs packages fonctionnels.

- `equipements.base` : Classes fondamentales (`Equipement`, `Ordinateur`, `Serveur` et `StationDeTravail`).
- `equipements.reseau` : Équipements spécifiques au réseau (`Switch`, `Routeur`).
- `equipements.peripheriques` : Composants externes (`Écran`, `Clavier`, `Souris`).
- `gestion` : Logique métier (`ParcInformatique`).
- `test` : Classes de validation et scénarios d'usage.

### 0.2 Arborescence finale des packages

```
projet(src)/
|
+-- equipements/
|   +-- base/
|       |   +-- Equipement.java
|       |   +-- Ordinateur.java
|       |   +-- Serveur.java
|       |   \-- StationDeTravail.java
|   +-- reseau/
|       |   +-- EquipementReseau.java
|       |   +-- Routeur.java
|       |   \-- Switch.java
|   \-- peripheriques/
|       +-- Clavier.java
|       +-- Ecran.java
|       \-- Souris.java
|
+-- gestion/
|   \-- ParcInformatique.java
|
\-- test/
    \-- (classes de test)
```

# 1 Classe de Base et Premier Héritage

## Les Fondamentaux

L'héritage définit une classe générale (**Mère**) avec les caractéristiques communes (comme *Equipement*) et des classes spécialisées (**Filles**) (comme *Ordinateur* ou *Serveur*), qui ajoutent ou adaptent des comportements spécifiques.

Cette relation représente un lien "**EST-UN**" : chaque classe fille est un type particulier de la classe mère.

- Un *Ordinateur* **EST UN** *Equipement*.
- Un *Switch* **EST UN** *EquipementReseau*.

## Concepts Clés

- **extends** : Mot-clé pour hériter d'une classe mère.
- **super** : Permet d'accéder aux constructeurs ou méthodes de la classe mère.
- **protected** : Visibilité accessible aux classes filles (plus restrictive que public mais moins que private).
- **Polymorphisme** : Traiter des objets différents (ex. *Ordinateur*, *Serveur*) de manière uniforme via leur classe commune.

## Exercice 1.1 : Classe mère Equipement

1. Créez le package `equipements.base` dans votre dossier source.
2. Créez la classe `Equipement` dans le package `equipements.base`.
3. Déclarez les attributs privés suivants : `marque` (String), `modele` (String), `numeroSerie` (String), `prixAchat` (double) et `anneeAchat` (int).
4. Créez un constructeur prenant tous les paramètres :
  - Initialisez chaque attribut.
  - Validez l'année : elle doit être comprise entre 2000 et l'année courante. Sinon, utilisez une valeur par défaut.

**Obtenir l'année courante :** Utilisez `Year.now().getValue()` pour obtenir l'année actuelle sous forme d'entier. N'oubliez pas d'importer le package `java.time.Year`.

**Exemple :** `int anneeCourante = Year.now().getValue();`

5. Ajoutez des **accesseurs** pour tous les attributs.

6. Implémentez la méthode `calculerAge()` : elle doit retourner l'âge de l'équipement en calculant la différence entre l'année courante et `anneeAchat`.
7. Redéfinissez la méthode `toString()` pour retourner une chaîne formatée représentant l'objet.

### Exercice 1.2 : Première classe fille - Ordinateur

---

1. Créez la classe `Ordinateur` qui hérite de `Equipement` dans le package `equipements.base`.
2. **Attributs supplémentaires (private)** : `processeur (String)`, `ram (int)` en Go, `stockage (int)` en Go, `typeOS (String)`.
3. Créez un constructeur complet qui initialise tous les attributs de l'ordinateur, y compris ceux hérités de `Equipement` et les attributs spécifiques à `Ordinateur`.
4. Implémentez la méthode `verifierConfigMinimale(int ramMin, int stockageMin)` : elle retourne `true` si l'ordinateur satisfait les valeurs minimales de RAM et de stockage passées en paramètres.
5. Redéfinissez la méthode `toString()` pour inclure les spécifications de l'ordinateur.

### Exercice 1.3 : Deuxième classe fille - Serveur

---

1. Créez la classe `Serveur` qui hérite de `Equipement` dans le package `equipements.base`.
2. **Attributs supplémentaires (private)** : `nombreCoeurs (int)`, `ram (int)`, `capaciteStockage (int)`, `systemeRaid (boolean)`, `estEnProduction (boolean)`.
3. Créez un constructeur complet qui initialise tous les attributs du serveur, y compris ceux hérités de `Equipement` et les attributs spécifiques à `Serveur`.
4. Implémentez les méthodes spécifiques au serveur :
  - `demarrer()` : met `estEnProduction` à `true`.
  - `arreter()` : met `estEnProduction` à `false`.
  - `afficherEtat()` : affiche si le serveur est actuellement en production.
5. Redéfinissez la méthode `toString()` pour inclure les spécifications du serveur.

### Exercice 1.4 : Tests unitaires

---

Créez le package `test` et la classe `TestEquipements` contenant une méthode `main` :

1. Créez plusieurs objets `Ordinateur` et `Serveur` avec des configurations variées (différentes RAM, stockage, processeurs, etc.).

**Astuce :** Vous pouvez créer et initialiser un tableau d'ordinateurs/serveurs en une seule ligne :

Exemple d'initialisation

```
Ordinateur[] ordinateurs = { new Ordinateur( "Dell", "XPS 15", "SN12345", 1500, 2022, "Intel i7", 16, 512, "Windows 11" ),
                             new Ordinateur( "Apple", "MacBook Pro", "SN67890", 2500, 2021, "M1 Pro", 32, 1024, "macOS Ventura" )
};
```

2. Pour chaque objet :
  - Affichez ses informations complètes via la méthode `toString()`.
  - Testez `calculerAge()` pour connaître l'ancienneté de chaque équipement.
  - Pour les ordinateurs, vérifiez si la configuration répond à un minimum requis en utilisant `verifierConfigMinimale(ramMin, stockageMin)`. Les valeurs `ramMin` et `stockageMin` doivent être demandées à l'utilisateur via une saisie au clavier.
  - Pour les serveurs, testez les méthodes `demarrer()`, `arreter()` et `afficherEtat()` pour simuler un cycle de mise en production.
3. Ajoutez des `System.out.println()` dans les constructeurs pour visualiser l'ordre d'initialisation, afin de vérifier que la classe mère (`Equipement`) est initialisée avant la classe fille (`Ordinateur` ou `Serveur`).

## 2 Redéfinition de Méthodes `equals()` et `hashCode()`

### Les Fondamentaux

Nous avons déjà redéfini la méthode `toString()` afin d'obtenir une représentation textuelle claire de nos objets. Dans cette partie, nous allons redéfinir deux autres méthodes importantes héritées de la classe `Object` :

- **`equals()`** : méthode que vous connaissez déjà, utilisée pour comparer deux objets selon leur **contenu** plutôt que leur référence mémoire.
- **`hashCode()`** : méthode qui génère un code numérique associé à un objet. Ce code est utilisé par certaines structures de données comme `HashMap`<sup>1</sup>, `HashSet`<sup>2</sup>, ou `Hashtable`<sup>3</sup>

1. Structure de données qui associe une *clé* à une *valeur*. L'accès aux valeurs se fait très rapidement grâce au code de hachage de la clé.

2. Collection qui stocke des éléments sans doublons. Le `hashCode()` est utilisé pour vérifier efficacement si un élément est déjà présent.

3. Ancienne structure similaire à `HashMap`, historiquement synchronisée. Elle est aujourd'hui moins

afin d'organiser et retrouver efficacement les objets.

**À noter :** Lorsque la méthode `equals()` est redéfinie, il est recommandé de redéfinir également `hashCode()` afin de garantir un comportement cohérent lors de l'utilisation d'objets dans ces structures de données.

Dans notre système de gestion d'équipements, l'identité d'un équipement est définie par son `numeroSerie`.

### Exercice 2.1 : Redéfinition des méthodes `equals()` et `hashCode()`

Dans la classe `Equipement` :

1. Redéfinissez la méthode `equals(Object obj)` :
  - Deux objets `Equipement` sont considérés égaux s'ils possèdent le même `numeroSerie`.
2. Redéfinissez la méthode `hashCode()` :
  - La méthode doit retourner le **code de hachage** du `numeroSerie`, afin que deux équipements avec le même numéro de série aient le même code de hachage.

**Note :** Dans les classes `Ordinateur` et `Serveur`, il n'est pas nécessaire de redéfinir `equals()` si l'unicité des objets est déjà définie par le `numeroSerie` dans la classe mère `Equipement`.

### Exercice 2.2 : Méthodes finales

1. Dans la classe `Equipement`, ajoutez la méthode suivante :

```
public final String genererCodeInventaire() {  
    ...  
}
```

Cette méthode doit retourner un code d'inventaire généré au format `EQ-anneeAchat-numeroSerie`, par exemple `"EQ-2024-XY12"`.

2. Essayez ensuite de redéfinir cette méthode dans la classe `Ordinateur`.
3. Observez l'erreur de compilation générée par le mot-clé `final`.

**Rappel :** Une méthode déclarée `final` ne peut pas être redéfinie dans une classe fille garantissant ainsi un comportement fixe et sécurisé dans la conception d'une classe.

utilisée dans les applications modernes.

### Exercice 2.3 : Méthode `calculerValeurActuelle()`

Afin de simuler la perte de valeur du matériel informatique au fil du temps, nous allons implémenter une logique de dépréciation financière.

1. Dans la classe `Equipement`, implémentez la méthode :

```
public double calculerValeurActuelle()
```

Cette méthode doit appliquer une dépréciation standard de **20% par an** sur le prix d'achat.

2. Dans la classe `Ordinateur`, redéfinissez cette méthode pour appliquer une dépréciation de **25% par an**, représentant une obsolescence plus rapide.
3. Dans la classe `Serveur`, redéfinissez cette méthode pour appliquer une dépréciation de **15% par an**, car ce matériel est généralement plus durable.

### Exercice 2.4 : Tests des nouvelles méthodes

Dans la classe `TestEquipements` :

1. Créez deux objets `Equipement` possédant le même `numeroSerie`.
2. Utilisez la méthode `equals()` pour vérifier si les deux objets sont considérés comme identiques :

```
System.out.println(e1.equals(e2));
```

3. Créez également :
  - un `Ordinateur`
  - un `Serveur`achetés la même année et possédant le même prix d'achat.
4. Observez la différence de dépréciation entre les deux types d'équipements.

## 3 Packages Multiples et Gestion de la Visibilité

### Rappel

La visibilité `protected` est conçue pour l'héritage : elle permet aux classes filles, même situées dans d'autres packages, d'accéder aux membres de la classe parent. Elle accorde également l'accès à toutes les classes situées dans le même package.

### Exercice 3.1 : Package `equipements.reseau`

Créez le package `equipements.reseau` pour séparer les concepts réseau.

1. Créez la classe `EquipementReseau` qui hérite de `Equipement`.
  - Déclarez les attributs **privés** suivants : `nombrePorts (int)`, `vitesseMax (int)` en Gbps, `adresseIP (String)`.
  - Créez un constructeur permettant d'initialiser ces attributs ainsi que ceux hérités de `Equipement`.
  - Ajoutez les **accesseurs** et **mutateurs** pour ces attributs.
  - Redéfinissez la méthode `toString()` afin d'inclure les informations spécifiques aux équipements réseau.
2. Créez la classe `Switch` qui hérite de `EquipementReseau`.
  - Déclarez l'attribut **privé** `supporteVLAN (boolean)`.
  - Créez un constructeur permettant d'initialiser cet attribut ainsi que ceux hérités de `EquipementReseau`.
  - Ajoutez les **accesseurs** et **mutateurs** nécessaires.
  - Redéfinissez la méthode `toString()` pour inclure les informations spécifiques au switch.
3. Créez la classe `Routeur` qui hérite de `EquipementReseau`.
  - Déclarez l'attribut **privé** `nombreInterfacesWAN (int)`.
  - Créez un constructeur permettant d'initialiser cet attribut ainsi que ceux hérités de `EquipementReseau`.
  - Ajoutez les **accesseurs** et **mutateurs** nécessaires.
  - Redéfinissez la méthode `toString()` pour inclure les informations spécifiques au routeur.

### Exercice 3.2 : Expérimentation avec `protected`

1. Dans `Equipement` (package `base`), changez la visibilité de `prixAchat` de `private` à `protected`.
2. Dans la classe `Routeur` (package `reseau`), essayez d'accéder directement à l'attribut `prixAchat` via `this.prixAchat`. Par exemple, vous pouvez ajouter à la fin du constructeur une instruction comme `System.out.println(this.prixAchat);` afin d'afficher sa valeur et vérifier que l'accès fonctionne.
3. Créez une méthode `protected void appliquerRemise(double pourcentage)` dans `Equipement`. Cette méthode doit appliquer une remise sur le `prixAchat` de l'équipement en fonction du pourcentage passé en paramètre et affiche le nouveau `prixAchat`.
4. Appelez cette méthode depuis un constructeur d'une classe fille, que ce soit dans le même package ou dans un autre. Par exemple, affichez d'abord le prix de l'équipement, puis appelez la méthode `appliquerRemise(...)` et affichez ensuite le nouveau prix pour observer l'effet de la remise.



### Exercice 3.3 : Test de visibilité

Dans la classe `TestEquipements` :

1. Créez deux objets : un `Routeur` et un `Switch`.
2. Affichez les informations initiales de ces deux nouveaux équipements via `toString()`.
3. Testez l'exécution et observez si la modification du `prixAchat` s'effectue correctement.
4. **Cas d'erreur** : Toujours dans la classe `TestEquipements`, essayez d'appeler la méthode `appliquerRemise()` depuis un objet de type `Equipement` ou d'une de ses classes filles. Observez que le compilateur refuse l'accès, car ici la classe `TestEquipements` n'est ni une sous-classe de `Equipement` ni située dans le même package, illustrant ainsi la protection offerte par la visibilité `protected`.

## 4 Polymorphisme et Gestion Centralisée

### Rappel

Le polymorphisme permet de manipuler des objets de types différents (`Ordinateur`, `Routeur`, `Switch`, etc.) via leur type commun (`Equipement`).

### Exercice 4.1 : Gestion de Parc

Créez le package `gestion` et la classe `ParcInformatique`.

Dans la classe `ParcInformatique` :

1. Définissez deux attributs :
  - un tableau d'équipements `Equipement[] inventaire` pour stocker les équipements du parc,
  - un entier `nbEq` pour suivre le nombre actuel d'équipements dans le tableau.
2. Ajoutez un constructeur qui prend en paramètre la capacité maximale du tableau et initialise `inventaire` avec cette taille.
3. Définissez la méthode `public void ajouterEquipement(Equipement e)` : elle ajoute un équipement au premier emplacement disponible du tableau.
4. Implémentez la méthode `public void afficherInventaireComplet()` : elle parcourt le tableau et affiche les informations de chaque équipement à l'aide de `toString()`.
5. Ajoutez la méthode `public double calculerValeurTotaleParc()` : elle parcourt le tableau et retourne la somme des valeurs actuelles de tous les équipements.

## Exercice 4.2 : Usage de instanceof

### Rappel : instanceof et conversions de type

En Java, lorsqu'un objet d'une classe fille est stocké dans une variable de type parent (par exemple un `Serveur` dans une variable `Equipement`), on parle d'**upcast**. Cette conversion est **automatique** et ne nécessite pas de cast explicite.

En revanche, pour accéder aux méthodes spécifiques d'une classe fille, il peut être nécessaire de récupérer le type réel de l'objet. On utilise alors l'opérateur `instanceof` pour vérifier son type :

#### Exemple

```
if (e instanceof Serveur) {  
    Serveur s = (Serveur) e;  
    s.afficherEtat();  
}
```

### Principe :

- Upcast : automatique (ex. `Serveur` → `Equipement`)
- Downcast : nécessite une vérification préalable avec `instanceof`

Ajoutez des méthodes de filtrage dans `ParcInformatique` :

1. `public void compterParType()` : parcourt le tableau `inventaire` et affiche le nombre d'`Ordinateur`, de `Serveur`, etc. Utilisez `instanceof` pour identifier le type réel de chaque équipement.
2. `public void demarrerTousLesServeurs()` : parcourt le tableau et, pour chaque objet de type `Serveur` (identifié avec `instanceof`), appelle directement la méthode `demarrer()` pour le mettre en production.

## Exercice 4.3 : Tests d'intégration

Dans le package `test`, créez une classe nommée `TestParcInformatique`.

1. Créez un parc.
2. Créez un tableau d'`Equipement` contenant différents types d'équipements : 3 `Ordinateur`, 2 `Serveur` et 1 `Switch`.
3. Ajoutez les équipements de ce tableau au parc informatique.
4. Vérifiez que `afficherInventaireComplet()` affiche bien les détails spécifiques de chaque type.
5. Testez le calcul de la valeur totale.

6. Testez le démarrage global des serveurs.

## 5 Composition et Extension

### Rappel : Composition vs Héritage

Un écran **N'EST PAS** un ordinateur. Un ordinateur **A UN** écran. La composition permet de relier des objets sans créer de relation d'héritage, enrichissant notre modèle objet.

### Exercice 5.1 : Création de périphériques

Créez le package `equipements.peripheriques` dans votre projet. Dans ce package, vous allez définir plusieurs classes représentant des périphériques pouvant être associés à un ordinateur.

1. Créez la classe `Ecran`.
  - Déclarez les attributs privés suivants : `taille` en pouces (`int`), `resolution` (`String`) et `connectique` (`String`).
  - Créez un constructeur permettant d'initialiser ces attributs.
  - Ajoutez les **accesseurs** et **mutateurs** nécessaires.
  - Redéfinissez la méthode `toString()` afin d'afficher les caractéristiques de l'écran.
2. Créez la classe `Clavier`.
  - Déclarez les attributs privés suivants : `type` (`String`, par exemple mécanique ou membrane) et `layout` (`String`, par exemple AZERTY ou QWERTY).
  - Créez un constructeur pour initialiser ces attributs.
  - Ajoutez les **accesseurs**, **mutateurs** et la méthode `toString()`.
3. Créez la classe `Souris`.
  - Déclarez les attributs privés suivants : `dpi` (`int`) et `sansFil` (`boolean`).
  - Créez un constructeur permettant d'initialiser ces attributs.
  - Ajoutez les **accesseurs**, **mutateurs** et la méthode `toString()`.

### Exercice 5.2 : Intégration des périphériques dans la classe `Ordinateur`

Nous allons maintenant intégrer ces périphériques à la classe `Ordinateur`.

1. Dans la classe `Ordinateur`, ajoutez des attributs permettant de représenter les périphériques connectés :
  - `private Ecran ecran;`

- `private Clavier clavier;`
  - `private Souris souris;`
2. Ajoutez des méthodes permettant d'associer ces périphériques à l'ordinateur :
    - `void attacherEcran(Ecran e)`
    - `void attacherClavier(Clavier c)`
    - `void attacherSouris(Souris s)`
  3. Mettez à jour la méthode `toString()` afin d'afficher également les informations des périphériques connectés lorsque ceux-ci sont présents.

---

### Exercice 5.3 : Classe spécialisée `StationDeTravail`

---

1. Créez la classe `StationDeTravail` dans le package `equipements.base`. Cette classe doit **hériter de** `Ordinateur`.
2. Déclarez les attributs privés suivants :
  - `carteGraphique (String)`
  - `usage (String)` : par exemple *Design*, *3D*, *Développement*.
3. Créez un constructeur permettant d'initialiser ces attributs ainsi que ceux hérités de la classe `Ordinateur`.
4. Redéfinissez la méthode `calculerValeurActuelle()` afin d'appliquer une dépréciation plus faible que pour un ordinateur classique, soit 15%.
5. Ajoutez une méthode `public boolean verifierConfiguration()` qui retourne un `boolean` indiquant si la configuration de la station de travail est adaptée à l'usage prévu.
  - Si l'usage est "3D" ou "Design" :
    - La carte graphique doit être définie (non nulle et non vide).
    - La RAM doit être au moins de 32 Go.
  - Si l'usage est "Développement" :
    - La RAM doit être au moins de 16 Go.
  - Pour tout autre usage, la méthode doit retourner `true`.

---

### Exercice 5.4 : Test final

---

1. Dans la classe `TestParcInformatique`, créez un objet de la classe `StationDeTravail` avec une configuration avancée.
2. Créez également plusieurs périphériques (`Ecran`, `Clavier`, `Souris`) et associez-les à cette station de travail.

3. Ajoutez cette station de travail dans le parc informatique.
4. Exécutez le programme et vérifiez que toutes les informations s'affichent correctement lors de l'appel de la méthode `afficherInventaireComplet()`.

#### Aller plus loin

- Testez différentes configurations de périphériques ou de stations de travail pour observer le comportement du parc.
- Vous pouvez personnaliser le scénario de test selon vos besoins.
- Générez la **Javadoc** pour vos classes afin de documenter vos méthodes et attributs.